

SECTION 6

SUPERVISOR (PRIVILEGED) INSTRUCTIONS

This section contains information about the supervisor privileged instructions for the M68000 family. Each instruction is described in detail, and the instruction descriptions are arranged in alphabetical order by instruction mnemonic.

Any differences within the M68000 family of instructions are identified in the instruction. If an instruction only applies to a certain processor or processors, the processor(s) that the instruction pertains to is identified under the title of the instruction. For example:

Invalidate Cache Lines (MC68040)

All references to the MC68000, MC68020, and MC68030 include references to the corresponding embedded controllers, MC68EC000, MC68EC020, and MC68EC030. All references to the MC68040 include the MC68LC040 and MC68EC040. This applies throughout this section unless otherwise specified.

If the instruction applies to all the M68000 family but a processor or processors may use a different instruction field, instruction format, etc., the differences will be identified within the paragraph. For example:

MC68020, MC68030 and MC68040 only

(bd,An,Xn)*	110	reg. number: An	(bd,PC,Xn)*	—	—
-------------	-----	-----------------	-------------	---	---

*Can be used with CPU32 processo

The following instructions are listed separately for each processor due to the many differences involved within the instruction:

PFLUSH	Flush ATC Entries
PMOVE	Move PMMU Register
PTEST	Test Logical Address

Appendix A Processor Instruction Summary provides a listing of all processors and the instructions that apply to them for quick reference.



ANDI to SR

AND Immediate to the Status Register (M68000 Family)

ANDI to SR

Operation: If Supervisor State
Then Source L SR → SR
ELSE TRAP

Assembler Syntax: ANDI # < data > ,SR

Attributes: size = (word)

Description: Performs an AND operation of the immediate operand with the contents of the status register and stores the result in the status register. All implemented bits of the status register are affected.

Condition Codes:

X	N	Z	V	C
*	*	*	*	*

- X—Cleared if bit 4 of immediate operand is zero; unchanged otherwise.
- N—Cleared if bit 3 of immediate operand is zero; unchanged otherwise.
- Z—Cleared if bit 2 of immediate operand is zero; unchanged otherwise.
- V—Cleared if bit 1 of immediate operand is zero; unchanged otherwise.
- C—Cleared if bit 0 of immediate operand is zero; unchanged otherwise.

Instruction Format:

5	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	1	0	0	1	1	1	1	1	0	0
16-BIT WORD DATA															

CINV

Invalidate Cache Lines (MC68040, MC68LC040)

CINV

Operation: If Supervisor State
Then Invalidate Selected Cache Lines
ELSE TRAP

Assembler

Syntax: CINVL < caches > ,(An)
CINVP < caches > ,(An)
CINVA < caches >

Where < caches > specifies the instruction cache, data cache, both caches, or neither cache.

Attributes: Unsized

Description: Invalidates selected cache lines. The data cache, instruction cache, both caches, or neither cache can be specified. Any dirty data in data cache lines that invalidate are lost; the CPUSH instruction must be used when dirty data may be contained in the data cache.

Specific cache lines can be selected in three ways:

1. CINVL invalidates the cache line (if any) matching the physical address in the specified address register.
2. CINVP invalidates the cache lines (if any) matching the physical memory page in the specified address register. For example, if 4K-byte page sizes are selected and An contains \$12345000, all cache lines matching page \$12345000 invalidate.
3. CINVA invalidates all cache entries.

Condition Codes:

Not affected.



CINV

Invalidate Cache Lines
(MC68040, MC68LC040)

CINV

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	1	0	0	CACHE		0	SCOPE		REGISTER		

Instruction Fields:

- Cache field—Specifies the Cache.
 - 00—No Operation
 - 01—Data Cache
 - 10—Instruction Cache
 - 11—Data and Instruction Caches
- Scope field—Specifies the Scope of the Operation.
 - 00—Illegal (causes illegal instruction trap)
 - 01—Line
 - 10—Page
 - 11—All
- Register field—Specifies the address register for line and page operations. For line operations, the low-order bits 3–0 of the address are don't cares. Bits 11–0 or 12–0 of the address are don't care for 4K-byte or 8K-byte page operations, respectively.

cpRESTORE

Coprocessor
Restore Functions
(MC68020, MC68030)

cpRESTORE

Operation:

If Supervisor State
Then Restore Internal State of Coprocessor
ELSE TRAP

Assembler
Syntax:

cpRESTORE < ea >

Attributes:

Unsize

Description:

Restores the internal state of a coprocessor usually after it has been saved by a preceding cpSAVE instruction.

Condition Codes:

Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			1	0	1	EFFECTIVE ADDRESS MODE REGISTER					

cpRESTORE

Coprocessor Restore Functions (MC68020, MC68030)

cpRESTORE

Instruction Fields:

Coprocessor ID field—Identifies the coprocessor that is to be restored. Coprocessor ID of 000 results in an F-line exception for the MC68030.

Effective Address field—Specifies the location where the internal state of the coprocessor is located. Only postincrement or control addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	111	100
(An) +	011	reg. number:An			
— (An)	—	—			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	111	010
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn], od)	110	reg. number:An	([bd,PC,Xn], od)	111	011
([bd,An],Xn,od)	110	reg.number:An	([bd,PC],Xn, od)	111	011

NOTE

If the format word returned by the coprocessor indicates “come again”, pending interrupts are not serviced.

cpSAVE

Coprocessor Save Function

(MC68020, MC68030)

cpSAVE

Operation:

If Supervisor State

Then Save Internal State of Coprocessor

ELSE TRAP

Assembler Syntax:

cpSAVE < ea >

Attributes:

Unsize

Description:

Saves the internal state of a coprocessor in a format that can be restored by a cpRESTORE instruction.

Condition Codes:

Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			1	0	0	EFFECTIVE ADDRESS					
											MODE			REGISTER	

Instruction Fields:

Coprocessor ID field—Identifies the coprocessor for this operation. Coprocessor ID of 000 results in an F-line exception for the MC68030.

Effective Address field—Specifies the location where the internal state of the coprocessor is to be saved. Only predecrement or control alterable addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	—	—			
— (An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn], od)	110	reg. number:An	([bd,PC,Xn],od)	—	—
([bd,An],Xn, od)	110	reg. number:An	([bd,PC],Xn, od)	—	—



CPUSH

Push and Invalidate Cache Lines
(MC68040, MC68LC040)

CPUSH

Operation: If Supervisor State
Then If Data Cache
Then Push Selected Dirty Data Cache Lines
Invalidate Selected Cache Lines
ELSE TRAP

Assembler Syntax: CPUSHL < caches > ,(An)
CPUSHP < caches > ,(An)
CPUSHA < caches >

Where < caches > specifies the instruction cache, data cache, both caches, or neither cache.

Attributes: Unsized

Description: Pushes and then invalidates selected cache lines. The DATA cache, instruction cache, both caches, or neither cache can be specified. When the data cache is specified, the selected data cache lines are first pushed to memory (if they contain dirty DATA) and then invalidated. Selected instruction cache lines are invalidated.

Specific cache lines can be selected in three ways:

1. CPUSHL pushes and invalidates the cache line (if any) matching the physical address in the specified address register.
2. CPUSHP pushes and invalidates the cache lines (if any) matching the physical memory page in the specified address register. For example, if 4K-byte page sizes are selected and An contains \$12345000, all cache lines matching page \$12345000 are selected.
3. CPUSHA pushes and invalidates all cache entries.

Condition Codes:

Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	1	0	0	CACHE		1	SCOPE		REGISTER		

CPUSH

Push and Invalidate Cache Lines (MC68040, MC68LC040)

CPUSH

Instruction Fields:

Cache field—Specifies the Cache.

- 00—No Operation
- 01—Data Cache
- 10—Instruction Cache
- 11—Data and Instruction Caches

Scope field—Specifies the Scope of the Operation.

- 00—Illegal (causes illegal instruction trap)
- 01—Line
- 10—Page
- 11—All

Register field—Specifies the address register for line and page operations. For line operations, the low-order bits 3–0 of the address are don't care. Bits 11–0 or 12–0 of the address are don't care for 4K-byte or 8K-byte page operations, respectively.



EORI
to SR

Exclusive-OR Immediate to the Status Register
(M68000 Family)

EORI
to SR

Operation: If Supervisor State
Then Source $\oplus$ SR $\rightarrow$ SR
ELSE TRAP

Assembler Syntax: EORI # < data > ,SR

Attributes: Size = (Word)

Description: Performs an exclusive-OR operation on the contents of the status register using the immediate operand and stores the result in the status register. All implemented bits of the status register are affected.

Condition Codes:

X	N	Z	V	C
*	*	*	*	*

- X—Changed if bit 4 of immediate operand is one; unchanged otherwise.
- N—Changed if bit 3 of immediate operand is one; unchanged otherwise.
- Z—Changed if bit 2 of immediate operand is one; unchanged otherwise.
- V—Changed if bit 1 of immediate operand is one; unchanged otherwise.
- C—Changed if bit 0 of immediate operand is one; unchanged otherwise.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	0	1	0	0	1	1	1	1	1	0	0
16-BIT WORD DATA															

FRESTORE

**Restore Internal
Floating-Point State**
(MC68881, MC68882, MC68040 only)

FRESTORE

Operation: If in Supervisor State
Then FPU State Frame → Internal State
ELSE TRAP

Assembler

Syntax: FRESTORE < ea >

Attributes: Unsized

Description: Aborts the execution of any floating-point operation in progress and loads a new floating-point unit internal state from the state frame located at the effective address. The first word at the specified address is the format word of the state frame. It specifies the size of the frame and the revision number of the floating-point unit that created it. A format word is invalid if it does not recognize the size of the frame or the revision number does not match the revision of the floating-point unit. If the format word is invalid, FRESTORE aborts, and a format exception is generated. If the format word is valid, the appropriate state frame is loaded, starting at the specified location and proceeding through higher addresses.

The FRESTORE instruction does not normally affect the programmer's model registers of the floating-point coprocessor, except for the NULL state size, as described below. It is only for restoring the user invisible portion of the machine. The FRESTORE instruction is used with the FMOVEM instruction to perform a full context restoration of the floating-point unit, including the floating- point data registers and system control registers. To accomplish a complete restoration, the FMOVEM instructions are first executed to load the programmer's model, followed by the FRESTORE instruction to load the internal state and continue any previously suspended operation.

FRESTORE

Restore Internal Floating-Point State (MC68881, MC68882, MC68040 only)

FRESTORE

The current implementation supports the following four state frames:

- NULL:** This state frame is 4 bytes long, with a format word of \$0000. An FRESTORE operation with this size state frame is equivalent to a hardware reset of the floating-point unit. The programmer's model is set to the reset state, with nonsignaling NaNs in the floating-point data registers and zeros in the floating-point control register, floating-point status register, and floating-point instruction address register. (Thus, it is unnecessary to load the programmer's model before this operation.)
- IDLE:** This state frame is 4 bytes long in the MC68040, 28 (\$1C) bytes long in the MC68881, and 60 (\$3C) bytes long in the MC68882. An FRESTORE operation with this state frame causes the floating-point unit to be restored to the idle state, waiting for the initiation of the next instruction, with no exceptions pending. The programmer's model is not affected by loading this type of state frame.
- UNIMP:** This state frame is generated only by the MC68040. It is 48 (\$30) bytes long. An FSAVE that generates this size frame indicates either an unimplemented floating-point instruction or only an E1 exception is pending. This frame is never generated when an unsupported data type exception is pending or an E3 exception is pending. If both E1 and E3 exceptions are pending, a BUSY frame is generated.
- BUSY:** This state frame is 96 (\$60) bytes long in the MC68040, 184 (\$B8) bytes long in the MC68881, and 216 (\$D8) bytes long in the MC68882. An FRESTORE operation with this size state frame causes the floating-point unit to be restored to the busy state, executing the instructions that were suspended by a previous FSAVE operation. The programmer's model is not affected by loading this type of state frame; however, the completion of the suspended instructions after the restore is executed may modify the programmer's model.

Floating-Point Status Register: Cleared if the state size is NULL; otherwise, not affected.

FRESTORE

Restore Internal Floating-Point State

(MC68881, MC68882, MC68040 only)

FRESTORE

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			1	0	1	EFFECTIVE ADDRESS MODE REGISTER					

Instruction Field:

Effective Address field—Determines the addressing mode for the state frame. Only postincrement or control addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	011	reg. number:An			
—(An)	—	—			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	111	010
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn], od)	110	reg. number:An	([bd,PC,Xn], od)	111	011
([bd,An],Xn, od)	110	reg. number:An	([bd,PC],Xn, od)	111	011

FSAVE

Save Internal Floating-Point State (MC68881, MC68882, MC68040 only)

FSAVE

Operation: If in Supervisor State
Then FPU Internal State → State Frame
ELSE TRAP

Assembler Syntax: FSAVE < ea >

Attributes: Unsized

Description: FSAVE allows the completion of any floating-point operation in progress for the MC68040. It saves the internal state of the floating-point unit in a state frame located at the effective address. After the save operation, the floating-point unit is in the idle state, waiting for the execution of the next instruction. The first word written to the state frame is the format word specifying the size of the frame and the revision number of the floating-point unit.

Any floating-point operations in progress when an FSAVE instruction is encountered can be completed before the FSAVE executes, saving an IDLE state frame. Execution of instructions already in the floating-point unit pipeline continues until completion of all instructions in the pipeline or generation of an exception by one of the instructions. An IDLE state frame is created by the FSAVE if no exceptions occurred; otherwise, a BUSY or an UNIMP stack frame is created.

FSAVE suspends the execution of any operation in progress and saves the internal state in a state frame located at the effective address for the MC68881/MC68882. After the save operation, the floating-point coprocessor is in the idle state, waiting for the execution of the next instruction. The first word written to the state frame is the format word, specifying the size of the frame and the revision number of the floating-point coprocessor. The microprocessor unit initiates the FSAVE instruction by reading the floating-point coprocessor save CIR. The floating-point coprocessor save CIR is encoded with a format word that indicates the appropriate action to be taken by the main processor. The current implementation of the floating-point coprocessor always returns one of five responses in the save CIR:

Value	Definition
\$0018	Save NULL state frame
\$0118	Not ready, come again
\$0218	Illegal, take format exception
\$XX18	Save IDLE state frame
\$XXB4	Save BUSY state frame

NOTE: XX is the floating-point coprocessor version number.

FSAVE

Save Internal Floating-Point State (MC68881, MC68882, MC68040 only)

FSAVE

The not ready format word indicates that the floating-point coprocessor is not prepared to perform a state save and that the microprocessor unit should process interrupts, if necessary, and re-read the save CIR. The floating-point coprocessor uses this format word to cause the main processor to wait while an internal operation completes, if possible, to allow an IDLE frame rather than a BUSY frame to be saved. The illegal format word aborts an FSAVE instruction that is attempted while the floating-point coprocessor executes a previous FSAVE instruction. All other format words cause the microprocessor unit to save the indicated state frame at the specified address. For state frame details see state frames in the appropriate user's manual.

The following state frames apply to both the MC68040 and the MC68881/MC68882.

- NULL:** This state frame is 4 bytes long. An FSAVE instruction that generates this state frame indicates that the floating-point unit state has not been modified since the last hardware reset or FRESTORE instruction with a NULL state frame. This indicates that the programmer's model is in the reset state, with nonsignaling NaNs in the floating-point data registers and zeros in the floating-point control register, floating-point status register, and floating-point instruction address register. (Thus, it is not necessary to save the programmer's model.)
- IDLE:** This state frame is 4 bytes long in the MC68040, 28 (\$1C) bytes long in the MC68881, and 60 (\$3C) bytes long in the MC68882. An FSAVE instruction that generates this state frame indicates that the floating-point unit finished in an idle condition and is without any pending exceptions waiting for the initiation of the next instruction.
- UNIMP:** This state frame is generated only by the MC68040. It is 48 (\$30) bytes long. An FSAVE that generates this size frame indicates either an unimplemented floating-point instruction or that only an E1 exception is pending. This frame is never generated when an unsupported data type exception or an E3 exception is pending. If both E1 and E3 exceptions are pending, a BUSY frame is generated.
- BUSY:** This state frame is 96 (\$60) bytes long in the MC68040, 184 (\$B8) bytes long in the MC68881, and 216 (\$D8) bytes long in the MC68882. An FSAVE instruction that generates this size state frame indicates that the floating-point unit encountered an exception while attempting to complete the execution of the previous floating-point instructions.



FSAVE Save Internal Floating-Point State FSAVE

(MC68881, MC68882, MC68040 only)

The FSAVE does not save the programmer’s model registers of the floating-point unit; it saves only the user invisible portion of the machine. The FSAVE instruction may be used with the FMOVEM instruction to perform a full context save of the floating-point unit that includes the floating-point data registers and system control registers. To accomplish a complete context save, first execute an FSAVE instruction to suspend the current operation and save the internal state, then execute the appropriate FMOVEM instructions to store the programmer’s model.

Floating-Point Status Register: Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			1	0	0	EFFECTIVE ADDRESS MODE REGISTER					

Instruction Field:

Effective Address field—Determines the addressing mode for the state frame. Only predecrement or control alterable addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	—	—			
—(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	—	—
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	—	—

MOVE from SR

Move from the Status Register
(MC68EC000, MC68010, MC68020,
MC68030, MC68040, CPU32)

MOVE from SR

Operation: If Supervisor State
Then SR → Destination
Else TRAP

Assembler Syntax: MOVE SR, < ea >

Attributes: Size = (Word)

Description: Moves the data in the status register to the destination location. The destination is word length. Unimplemented bits are read as zeros.

Condition Codes:
Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	0	1	1	EFFECTIVE ADDRESS					
										MODE		REGISTER			

MOVE from SR

Move from the Status Register
(MC68EC000, MC68010, MC68020,
MC68030, MC68040, CPU32)

MOVE from SR

Instruction Field:

Effective Address field—Specifies the destination location. Only data alterable addressing modes can be used as listed in the following tables:

Addressing Mode	Mode	Register
Dn	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An) +	011	reg. number:An
—(An)	100	reg. number:An
(d ₁₆ ,An)	101	reg. number:An
(d ₈ ,An,Xn)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
# < data >	—	—
(d ₁₆ ,PC)	—	—
(d ₈ ,PC,Xn)	—	—

MC68020, MC68030, and MC68040 only

(bd,An,Xn)*	110	reg. number:An
([bd,An,Xn] ,od)	110	reg. number:An
([bd,An],Xn ,od)	110	reg. number:An

(bd,PC,Xn)*	—	—
([bd,PC,Xn] ,od)	—	—
([bd,PC],Xn ,od)	—	—

*Available for the CPU32.

NOTE

Use the MOVE from CCR instruction to access only the condition codes.

MOVE to SR

Move to the Status Register (M68000 Family)

MOVE to SR

Operation: If Supervisor State
Then Source → SR
Else TRAP

Assembler Syntax: MOVE < ea > ,SR

Attributes: Size = (Word)

Description: Moves the data in the source operand to the status register. The source operand is a word, and all implemented bits of the status register are affected.

Condition Codes:
Set according to the source operand.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	1	0	1	1	EFFECTIVE ADDRESS					
										MODE		REGISTER			

MOVE to SR

Move to the Status Register (M68000 Family)

MOVE to SR

Instruction Field:

Effective Address field—Specifies the location of the source operand. Only data addressing modes can be used as listed in the following tables:

Addressing Mode	Mode	Register
Dn	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An) +	011	reg. number:An
—(An)	100	reg. number:An
(d ₁₆ ,An)	101	reg. number:An
(d ₈ ,An,Xn)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
# < data >	111	100
(d ₁₆ ,PC)	111	010
(d ₈ ,PC,Xn)	111	011

MC68020, MC68030, and MC68040 only

(bd,An,Xn)*	110	reg. number:An
([bd,An,Xn] ,od)	110	reg. number:An
([bd,An],Xn ,od)	110	reg. number:An

(bd,PC,Xn)*	111	011
([bd,PC,Xn] ,od)	111	011
([bd,PC],Xn ,od)	111	011

*Available for the CPU32.

MOVE USP

Move User Stack Pointer (M68000 Family)

MOVE USP

Operation: If Supervisor State
Then USP → An or An → USP
Else TRAP

Assembler Syntax: MOVE USP,An
MOVE An,USP

Attributes: Size = (Long)

Description: Moves the contents of the user stack pointer to or from the specified address register.

Condition Codes:
Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	1	1	1	0	0	1	1	0	dr	REGISTER		

Instruction Fields:

- dr field—Specifies the direction of transfer.
 - 0—Transfer the address register to the user stack pointer.
 - 1—Transfer the user stack pointer to the address register.
- Register field—Specifies the address register for the operation.



MOVEC

Move Control Register
(MC68010, MC68020, MC68030, MC68040, CPU32)

MOVEC

Operation: If Supervisor State
Then Rc → Rn or Rn → Rc
Else TRAP

Assembler MOVEC Rc,Rn
Syntax: MOVEC Rn,Rc

Attributes: Size = (Long)

Description: Moves the contents of the specified control register (Rc) to the specified general register (Rn) or copies the contents of the specified general register to the specified control register. This is always a 32-bit transfer, even though the control register may be implemented with fewer bits. Unimplemented bits are read as zeros.

Condition Codes:
Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	1	1	1	0	0	1	1	1	1	0	1	dr
A/D		REGISTER			CONTROL REGISTER										

Instruction Fields:

- dr field—Specifies the direction of the transfer.
0—Control register to general register.
1—General register to control register.
- A/D field—Specifies the type of general register.
0—Data Register
1—Address Register

MOVEC

Move Control Register (MC68010, MC68020, MC68030, MC68040, CPU32)

MOVEC

Register field—Specifies the register number.
Control Register field—Specifies the control register.

Hex ¹	Control Register
MC68010/MC68020/MC68030/MC68040/CPU32	
000	Source Function Code (SFC)
001	Destination Function Code (DFC)
800	User Stack Pointer (USP)
801	Vector Base Register (VBR)
MC68020/MC68030/MC68040	
002	Cache Control Register (CACR)
802	Cache Address Register (CAAR) ²
803	Master Stack Pointer (MSP)
804	Interrupt Stack Pointer (ISP)
MC68040/MC68LC040	
003	MMU Translation Control Register (TC)
004	Instruction Transparent Translation Register 0 (ITT0)
005	Instruction Transparent Translation Register 1 (ITT1)
006	Data Transparent Translation Register 0 (DTT0)
007	Data Transparent Translation Register 1 (DTT1)
805	MMU Status Register (MMUSR)
806	User Root Pointer (URP)
807	Supervisor Root Pointer (SRP)
MC68EC040 only	
004	Instruction Access Control Register 0 (IACR0)
005	Instruction Access Control Register 1 (IACR1)
006	Data Access Control Register 0 (DACR0)
007	Data Access Control Register 1 (DACR1)

- NOTES:
- Any other code causes an illegal instruction exception
 - For the MC68020 and MC68030 only.



MOVES

Move Address Space
(MC68010, MC68020, MC68030, MC68040, CPU32)

MOVES

Operation: If Supervisor State
Then Rn → Destination [DFC] or Source [SFC] → Rn
Else TRAP

Assembler Syntax: MOVES Rn, < ea >
MOVES < ea > ,Rn

Attributes: Size = (Byte, Word, Long)

Description: This instruction moves the byte, word, or long operand from the specified general register to a location within the address space specified by the destination function code (DFC) register, or it moves the byte, word, or long operand from a location within the address space specified by the source function code (SFC) register to the specified general register. If the destination is a data register, the source operand replaces the corresponding low-order bits of that data register, depending on the size of the operation. If the destination is an address register, the source operand is sign-extended to 32 bits and then loaded into that address register.

Condition Codes:
Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	1	1	0	SIZE		EFFECTIVE ADDRESS					
										MODE			REGISTER		
A/D	REGISTER			dr	0	0	0	0	0	0	0	0	0	0	0

MOVES

Move Address Space (MC68010, MC68020, MC68030, MC68040, CPU32)

MOVES

Instruction Fields:

Size field—Specifies the size of the operation.

00—Byte Operation

01—Word Operation

10—Long Operation

Effective Address field—Specifies the source or destination location within the alternate address space. Only memory alterable addressing modes can be used as listed in the following tables:

Addressing Mode	Mode	Register
Dn	—	—
An	—	—
(An)	010	reg. number:An
(An) +	011	reg. number:An
—(An)	100	reg. number:An
(d ₁₆ ,An)	101	reg. number:An
(d ₈ ,An,Xn)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
# < data >	—	—
(d ₁₆ ,PC)	—	—
(d ₈ ,PC,Xn)	—	—

MC68020, MC68030, and MC68040 only

(bd,An,Xn)*	110	reg. number:An
([bd,An,Xn] ,od)	110	reg. number:An
([bd,An],Xn ,od)	110	reg. number:An

(bd,PC,Xn)*	—	—
([bd,PC,Xn] ,od)	—	—
([bd,PC],Xn ,od)	—	—

*Available for the CPU32.

A/D field—Specifies the type of general register.

0—Data Register

1—Address Register

Register field—Specifies the register number.

dr field—Specifies the direction of the transfer.

0—From < ea > to general register.

1—From general register to < ea > .

MOVES

Move Address Space
(MC68010, MC68020, MC68030, MC68040, CPU32)

MOVES

NOTE

The value stored is undefined for either of the two following examples with the same address register as both source and destination.

```
MOVES.x An,(An) +  
MOVES.x An,D(An)
```

The current implementations of the MC68010, MC68020, MC68030, and MC68040 store the incremented or decremented value of An. Check the following code sequence to determine what value is stored for each case.

```
MOVEA.L #$1000,A0  
MOVES.L A0,(A0) +  
MOVES.L A0,D(A0)
```

Because the MC68040 implements a merged instruction and data space, the MC68040's integer unit into data references (SFC/DFC = 5 or 1) translates MOVES accesses to the OinstructionO address spaces (SFC/DFC = 6 or 2). The data memory unit handles these translated accesses as normal data accesses. If the access fails due to an ATC fault or a physical bus error, the resulting access error stack frame contains the converted function code in the TM field for the faulted access. To maintain cache coherency, MOVES accesses to write the OinstructionO address space must be preceded by invalidation of the instruction cache line containing the referenced location.

ORI
to SR

Inclusive-OR Immediate to the Status Register
(M68000 Family)

ORI
to SR

Operation: If Supervisor State
Then Source V SR → SR
Else TRAP

Assembler Syntax: ORI # < data > ,SR

Attributes: Size = (Word)

Description: Performs an inclusive-OR operation of the immediate operand and the status register’s contents and stores the result in the status register. All implemented bits of the status register are affected.

Condition Codes:

X	N	Z	V	C
*	*	*	*	*

- X—Set if bit 4 of immediate operand is one; unchanged otherwise.
- N—Set if bit 3 of immediate operand is one; unchanged otherwise.
- Z—Set if bit 2 of immediate operand is one; unchanged otherwise.
- V—Set if bit 1 of immediate operand is one; unchanged otherwise.
- C—Set if bit 0 of immediate operand is one; unchanged otherwise.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	1	1	1	1	1	0	0
16—BIT WORD DATA															



PBcc Branch on PMMU Condition (MC68851) PBcc

Operation: If Supervisor State
Then If cc True
Then (PC) + dn -> PC
Else TRAP

Assembler
Syntax: PBcc. < size > < label >

Attributes: Size = (Word, Long)

Description: If the specified paged memory management unit condition is met, execution continues at location (PC) + displacement. The displacement is a twos complement integer that counts the relative distance in bytes. The value in the program counter is the address of the displacement word(s). The displacement may be either 16 or 32 bits.

The condition specifier cc indicates the following conditions:

Table with 2 main sections. Each section has 3 columns: Specifier, Description, Condition Field. Left section lists set conditions (BS, LS, SS, AS, WS, IS, GS, CS). Right section lists clear conditions (BC, LC, SC, AC, WC, IC, GC, CC).

PMMU Status Register: Not affected.

Instruction Format:

Instruction format diagram showing bit positions 15 down to 0. Fields include SIZE, MC68851 CONDITION, 16-BIT DISPLACEMENT OR MOST SIGNIFICANT WORD OF 32-BITDISPLACEMENT, and LEAST SIGNIFICANT WORD OF 32-BIT DISPLACEMENT (IF NEEDED).

PBcc

**Branch on PMMU Condition
(MC68851)**

PBcc
Instruction Fields:

Size field—Specifies the size of the displacement.

0—Displacement is 16 bits.

1—Displacement is 32 bits.

MC68851 Condition field—Specifies the coprocessor condition to be tested. This field is passed to the MC68851, which provides directives to the main processor for processing this instruction.

Word Displacement field—The shortest displacement form for MC68851 branches is 16 bits.

Long-Word Displacement field—Allows a displacement larger than 16 bits.

PDBcc

Test, Decrement, and Branch (MC68851)

PDBcc

Operation: If Supervisor State
 Then If cc False
 Then (DnD1 → Dn; If Dn < > D1 then (PC) + d 3 PC)
 Else No Operation
 Else TRAP

Assembler

Syntax: PDBcc Dn, < label >

Attributes: Size = (Word)

Description: This instruction is a looping primitive of three parameters: an MC68851 condition, a counter (an MC68020 data register), and a 16-bit displacement. The instruction first tests the condition to determine if the termination condition for the loop has been met. If so, the main processor executes the next instruction in the instruction stream. If the termination condition is not true, the low-order 16 bits of the counter register are decremented by one. If the result is not D1, execution continues at the location specified by the current value of the program counter plus the sign-extended 16-bit displacement. The value of the program counter used in the branch address calculation is the address of the PDBcc instruction plus two.

The condition specifier cc indicates the following conditions:

Specifier	Description	Condition Field	Specifier	Description	Condition Field
BS	B set	000000	BC	B clear	000001
LS	L set	000010	LC	L clear	000011
SS	S set	000100	SC	S clear	000101
AS	A set	000110	AC	A clear	000111
WS	W set	001000	WC	W clear	001001
IS	I set	001010	IC	I clear	001011
GS	G set	001100	GC	G clear	001101
CS	C set	001110	CC	C clear	001111

PDBcc

Test, Decrement, and Branch
(MC68851)

PDBcc

PMMU Status Register:Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	1	0	0	1	COUNT REGISTER		
0	0	0	0	0	0	0	0	0	0	MC68851 CONDITION					
16-BIT DISPLACEMENT															

Instruction Fields:

- Register field—Specifies the data register in the main processor to be used as the counter.
- MC68851 Condition field—Specifies the MC68851 condition to be tested. This field is passed to the MC68851, which provides directives to the main processor for processing this instruction.
- Displacement field—Specifies the distance of the branch in bytes.

PFLUSH

Flush Entry in the ATC (MC68030 only)

PFLUSH

Operation: If Supervisor State
Then Invalidate ATC Entries for Destination Addresses
Else TRAP

Assembler PFLUSHA
Syntax: PFLUSH FC,MASK
PFLUSH FC,MASK, < ea >

Attributes: Unsized

Description: PFLUSH invalidates address translation cache entries. The instruction has three forms. The PFLUSHA instruction invalidates all entries. When the instruction specifies a function code and mask, the instruction invalidates all entries for a selected function code(s). When the instruction also specifies an < ea > , the instruction invalidates the page descriptor for that effective address entry in each selected function code.

The mask operand contains three bits that correspond to the three function code bits. Each bit in the mask that is set to one indicates that the corresponding bit of the FC operand applies to the operation. Each bit in the mask that is zero indicates a bit of FC and of the ignored function code. For example, a mask operand of 100 causes the instruction to consider only the most significant bit of the FC operand. If the FC operand is 001, function codes 000, 001, 010, and 011 are selected.

The FC operand is specified in one of the following ways:

1. Immediate—Three bits in the command word.
2. Data Register—The three least significant bits of the data register specified in the instruction.
3. Source Function Code (SFC) Register
4. Destination Function Code (DFC) Register

Condition Codes:

Not affected.

MMU Status Register:

Not affected.

PFLUSH

Flush Entry in the ATC
(MC68030 only)

PFLUSH

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS						
				MODE		0		0		MASK					REGISTER	
0	0	1				MODE		0	0	MASK			FC			

Instruction Fields:

Effective Address field—Specifies a control alterable address. The address translation cache entry for this address is invalidated. Valid addressing modes are in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	—	—			
—(An)	—	—			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	—	—
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	—	—

NOTE

The address field must provide the memory management unit with the effective address to be flushed from the address translation cache, not the effective address describing where the PFLUSH operand is located. For example, to flush the address translation cache entry corresponding to a logical address that is temporarily stored on top of the system stack, the instruction PFLUSH [(SP)] must be used since PFLUSH (SP) would invalidate the address translation cache entry mapping the system stack (i.e., the effective address passed to the memory management unit is the effective address of the system stack, not the effective address formed by the operand located on the top of the stack).



PFLUSH

Flush Entry in the ATC (MC68030 only)

PFLUSH

Mode field—Specifies the type of flush operation.

001—Flush all entries.

100—Flush by function code only.

110—Flush by function code and effective address.

Mask field—Mask for selecting function codes. Ones in the mask correspond to applicable bits; zeros are bits to be ignored. When mode is 001, mask must be 000.

FC field—Function code of entries to be flushed. If the mode field is 001, FC field must be 00000; otherwise:

10XXX — Function code is specified as bits XXX.

01DDD — Function code is specified as bits 2–0 of data register DDD.

00000 — Function code is specified as SFC register.

00001 — Function code is specified as DFC register.

PFLUSH

Flush ATC Entries (MC68040, MC68LC040)

PFLUSH

Operation: If Supervisor State
Then Invalidate Instruction and Data ATC Entries for Destination Address
Else TRAP

Assembler PFLUSH (An)
Syntax: PFLUSHN (An)
Syntax: PFLUSHA
Syntax: PFLUSHAN

Attributes: Unsized

Description: Invalidates address translation cache entries in both the instruction and data address translation caches. The instruction has two forms. The PFLUSHA instruction invalidates all entries. The PFLUSH (An) instruction invalidates the entry in each address translation cache which matches the logical address in An and the specified function code.

The function code for PFLUSH is specified in the destination function code register. Destination function code values of 1 or 2 will result in flushing of user address translation cache entries in both address translation caches; whereas, values of 5 or 6 will result in flushing of supervisor address translation cache entries. PFLUSH is undefined for destination function code values of 0, 3, 4, and 7 and may cause flushing of an unexpected entry.

The PFLUSHN and PFLUSHAN instructions have a global option specified and invalidate only nonglobal entries. For example, if only page descriptors for operating system code have the global bit set, these two PFLUSH variants can be used to flush only user address translation cache entries during task swaps.

Condition Codes:

Not affected.



PFLUSH

Flush ATC Entries
(MC68040, MC68LC040)

PFLUSH

Instruction Format:

Postincrement Source and Destination

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	1	0	1	0	0	0	OPMODE		REGISTER		

Instruction Fields:

Opmode field—Specifies the flush operation.

Opcode	Operation	Assembler Syntax
00	Flush page entry if not global	PFLUSHN (An)
01	Flush page entry	PFLUSH (An)
10	Flush all except global entries	PFLUSHAN
11	Flush all entries	PFLUSHA

Register field—Specifies the address register containing the effective address to be flushed when flushing a page entry.

PFLUSH

Flush ATC Entries
(MC68EC040)

PFLUSH

Operation: If Supervisor State
Then No Operation
Else TRAP

Assembler Syntax: PFLUSH (An)
PFLUSHN (An)

Attributes: Unsized

Description: This instruction should not be executed when using an MC68EC040. The PFLUSH encoding suspends operation of the MC68EC040 for an indefinite period of time and subsequently continues with no adverse effects.

Condition Codes:

Not affected.

Instruction Format:

Postincrement Source and Destination

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	1	0	1	0	0	0	OPMODE	REGISTER			

Instruction Fields:

Opmode field—Specifies the flush operation.

Opcode	Operation	Assembler Syntax
00	Flush page entry if not global	PFLUSHN (An)
01	Flush page entry	PFLUSH (An)
10	Flush all except global entries	PFLUSHAN
11	Flush all entries	PFLUSHA

Register field—Specifies the address register containing the effective address to be flushed when flushing a page entry.

PFLUSH PFLUSHA PFLUSHS

Invalidate Entries in the ATC
(MC68851)

PFLUSH PFLUSHA PFLUSHS

Operation: If Supervisor State
Then Address Translation Cache Entries For Destination Address
Are Invalidated
Else TRAP

Assembler Syntax: PFLUSHA
PFLUSH FC,MASK
PFLUSHS FC,MASK
PFLUSH FC,MASK, < ea >
PFLUSHS FC,MASK, < ea >

Attributes: Unsigned

Description: PFLUSHA invalidates all entries in the address translation cache.

PFLUSH invalidates a set of address translation cache entries whose function code bits satisfy the relation: (address translation cache function code bits and mask) = (FC and MASK) for all entries whose task alias matches the task alias currently active when the instruction is executed. With an additional effective address argument, PFLUSH invalidates a set of address translation cache entries whose function code satisfies the relation above and whose effective address field matches the corresponding bits of the evaluated effective address argument. In both of these cases, address translation cache entries whose SG bit is set will not be invalidated unless the PFLUSHS is specified.

The function code for this operation may be specified as follows:

1. Immediate—The function code is four bits in the command word.
2. Data Register—The function code is in the lower four bits of the MC68020 data register specified in the instruction.
3. Source Function Code (SFC) Register—The function code is in the CPU SFC register. Since the SFC of the MC68020 has only three implemented bits, only function codes \$0D\$7 can be specified in this manner.
4. Destination Function Code (DFC) Register—The function code is in the CPU DFC register. Since the DFC of the MC68020 has only three implemented bits, only function codes \$0D\$7 can be specified in this manner.

PFLUSH PFLUSHA PFLUSHS

Invalidate Entries in the ATC
(MC68851)

PFLUSH PFLUSHA PFLUSHS

PMMU Status Register: Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	0	1	MODE			0	MASK				FC				

Instruction Fields:

Effective Address field—Specifies an address whose page descriptor is to be flushed from (invalidated) the address translation cache. Only control alterable addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	—	—			
—(An)	—	—			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	—	—
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	—	—

PFLUSH
PFLUSHA
PFLUSHS

**Invalidate Entries in the ATC
(MC68851)**

PFLUSH
PFLUSHA
PFLUSHS

NOTE

The effective address field must provide the MC68851 with the effective address of the entry to be flushed from the address translation cache, not the effective address describing where the PFLUSH operand is located. For example, in order to flush the address translation cache entry corresponding to a logical address that is temporarily stored on the top of the system stack, the instruction PFLUSH [(SP)] must be used since PFLUSH (SP) would invalidate the address translation cache entry mapping the system stack (i.e., the effective address passed to the MC68851 is the effective address of the system stack, not the effective address formed by the operand located on the top of the stack).

Mode field—Specifies how the address translation cache is to be flushed.

- 001—Flush all entries.
- 100—Flush by function code only.
- 101—Flush by function code including shared entries.
- 110—Flush by function code and effective address.
- 111—Flush by function code and effective address including shared entries.

Mask field—Indicates which bits are significant in the function code compare. A zero indicates that the bit position is not significant; a one indicates that the bit position is significant. If mode = 001 (flush all entries), mask must be 0000.

FC field—Function code of address to be flushed. If the mode field is 001 (flush all entries), function code must be 00000; otherwise:

- 1DDDD — Function code is specified as four bits DDDD.
- 01RRR — Function code is contained in CPU data register RRR.
- 00000 — Function code is contained in CPU SFC register.
- 00001 — Function code is contained in CPU DFC register.

PFLUSHR

Invalidate ATC and RPT Entries (MC68851)

PFLUSHR

Operation: If Supervisor State
 Then RPT Entry (If Any) Matching Root Pointer Specified by < ea >
 Corresponding Address Translation Cache Entries Are Invalidated
 Else TRAP

Assembler

Syntax: PFLUSHR < ea >

Attributes: Unsized

Description: The quad word pointed to by < ea > is regarded as a previously used value of the CPU root pointer register. The root pointer table entry matching this CPU root pointer register (if any) is flushed, and all address translation cache entries loaded with this value of CPU root pointer register (except for those that are globally shared) are invalidated. If no entry in the root pointer table matches the operand of this instruction, no action is taken.

If the supervisor root pointer is not in use, the operating system should not issue the PFLUSHR command to destroy a task identified by the current CPU root pointer register. It should wait until the CPU root pointer register has been loaded with the root pointer identifying the next task until using the PFLUSHR instruction. At any time, execution of the PFLUSHR instruction for the current CPU root pointer register causes the current task alias to be corrupted.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		
1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0



PFLUSHR

Invalidate ATC and RPT Entries
(MC68851)

PFLUSHR

Instruction Field:

Effective Address field—Specifies the address of a previous value of the CPU root pointer register register. Only memory addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	111	100
(An) +	011	reg. number:An			
—(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	111	010
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	111	011
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	111	011

NOTE

The effective address usage of this instruction is different than that of other PFLUSH variants.

PLOAD

Load an Entry into the ATC (MC68030 only, MC68851)

PLOAD

Operation:	If Supervisor State Then Search Translation Table and Make Address Translation Cache Entry for Effective Address Else TRAP
Assembler Syntax:	PLOADR FC, < ea > PLOADW FC, < ea >
Attributes:	Unsize

Description: For the MC68851, PLOAD searches the translation table for a translation of the specified effective address. If one is found, it is flushed from the address translation cache, and an entry is made as if a bus master had run a bus cycle. Used and modified bits in the table are updated as part of the table search. The MC68851 ignores the logical bus arbitration signals during the flush and load phases at the end of this instruction. This prevents the possibility of an entry temporarily disappearing from the address translation cache and causing a false table search.

This instruction will cause a paged memory management unit illegal operation exception (vector \$39) if the E-bit of the translation control register is clear.

The function code for this operation may be specified to be:

1. Immediate—The function code is specified as four bits in the command word.
2. Data Register—The function code is contained in the lower four bits in the MC68020 data register specified in the instruction.
3. Source Function Code (SFC) Register—The function code is in the CPU SFC register. Since the SFC of the MC68020 has only three implemented bits, only function codes \$0D\$7 can be specified in this manner.
4. Destination Function Code (DFC) Register—The function code is in the CPU DFC register. Since the DFC of the MC68020 has only three implemented bits, only function codes \$0D\$7 can be specified in this manner.



PLOAD

Load an Entry into the ATC
(MC68030 only, MC68851)

PLOAD

For the MC68030, PLOAD searches the address translation cache for the specified effective address. It also searches the translation table for the descriptor corresponding to the specified effective address. It creates a new entry as if the MC68030 had attempted to access that address. Sets the used and modified bits appropriately as part of the search. The instruction executes despite the value of the E-bit in the translation control register or the state of the MMUDIS signal.

The < function code > operand is specified in one of the following ways:

- 1. Immediate—Three bits in the command word.
- 2. Data Register—The three least significant bits of the data register specified in the instruction.
- 3. Source Function Code (SFC) Register
- 4. Destination Function Code (DFC) Register

The effective address field specifies the logical address whose translation is to be loaded.

PLOADR causes U bits in the translation tables to be updated as if a read access had occurred. PLOADW causes U and M bits in the translation tables to be updated as if a write access had occurred.

PMMU Status Register: Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE		REGISTER			
0	0	1	0	0	0	R/ W	0	0	0	0	FC				

PLOAD

Load an Entry into the ATC
(MC68030 only, MC68851)

PLOAD

Instruction Fields:

Effective Address field—Specifies the logical address whose translation is to be loaded into the address translation cache. Only control alterable addressing modes are allowed as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	—	—			
—(An)	—	—			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	—	—
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	—	—

NOTE

The effective address field must provide the MC68851 with the effective address of the entry to be loaded into the address translation cache, not the effective address describing where the PLOAD operand is located. For example, to load an address translation cache entry to map a logical address that is temporarily stored on the system stack, the instruction PLOAD [(SP)] must be used since PLOAD (SP) would load an address translation cache entry mapping the system stack (i.e., the effective address passed to the MC68851 is the effective address of the system stack, not the effective address formed by the operand located on the top of the stack).

R/W field—Specifies whether the tables should be updated for a read or a write.
1—Read
0—Write



PLOAD

Load an Entry into the ATC (MC68030 only, MC68851)

PLOAD

FC field (MC68851)—Function code of address to load.

1DDDD — Function code is specified as four bits DDDD.

01RRR — Function code is contained in CPU data register RRR.

00000 — Function code is contained in CPU SFC register.

00001 — Function code is contained in CPU DFC register.

FC field (MC68030)—Function code of address corresponding to entry to be loaded.

10XXX — Function code is specified as bits XXX.

01DDD — Function code is specified as bits 2–0 of data register DDD.

00000 — Function code is specified as SFC register.

00001 — Function code is specified as DFC register.

PMOVE

Move to/from MMU Registers (MC68030 only)

PMOVE

Operation: If Supervisor State
Then (Source) → MRn or MRn → (Destination)

Assembler Syntax: PMOVE MRn, < ea >
PMOVE < ea > ,MRn
PMOVEFD < ea > ,MRn

Attributes: Size = (Word, Long, Quad)

Description: Moves the contents of the source effective address to the specified memory management unit register or moves the contents of the memory management unit register to the destination effective address.

The instruction is a quad-word (8 byte) operation for the CPU root pointer and the supervisor root pointer. It is a long-word operation for the translation control register and the transparent translation registers (TT0 and TT1). It is a word operation for the MMU status register.

The PMOVEFD form of this instruction sets the FD-bit to disable flushing the address translation cache when a new value loads into the supervisor root pointer, CPU root pointer, TT0, TT1 or translation control register (but not the MMU status register).

Writing to the following registers has the indicated side effects:

CPU Root Pointer—When the FD-bit is zero, it flushes the address translation cache. If the operand value is invalid for a root pointer descriptor, the instruction takes an memory management unit configuration error exception after moving the operand to the CPU root pointer.

Supervisor Root Pointer—When the FD-bit is zero, it flushes the address translation cache. If the operand value is invalid as a root pointer descriptor, the instruction takes an memory management unit configuration error exception after moving the operand to the supervisor root pointer.

Translation Control Register—When the FD-bit is zero, it flushes the address translation cache. If the E-bit = 1, consistency checks are performed on the PS and Tlx fields. If the checks fail, the instruction takes an memory management unit configuration exception after moving the operand to the translation control register. If the checks pass, the translation control register is loaded with the operand and the E-bit is cleared.

TT0, TT1—When the FD-bit is zero, it flushes the address translation cache. It enables or disables the transparent translation register according to the E-bit written. If the E-bit = 1, the transparent translation register is enabled. If the E-bit = 0, the register is disabled.



PMOVE Move to/from MMU Registers (MC68030 only) PMOVE

Condition Codes:

Not affected.

MMU Status Register:

Not affected (unless the MMU status register is specified as the destination operand).

Instruction Format:

SRP, CRP, and TC Registers

Table with 16 columns (15-0) and 3 rows. Row 1: 15=1, 14=1, 13=1, 12=1, 11=0, 10=0, 9=0, 8=0, 7=0, 6=0, 5-0: EFFECTIVE ADDRESS. Row 2: 5-0: MODE and REGISTER. Row 3: 15=0, 14=1, 13=0, 12-10: P-REGISTER, 9: R/W, 8: FD, 7=0, 6=0, 5=0, 4=0, 3=0, 2=0, 1=0, 0=0.

Instruction Fields:

Effective Address field—Specifies the memory location for the transfer. Only control alterable addressing modes can be used as in the following table:

Two tables side-by-side showing Addressing Mode, Mode, and Register. Left table includes modes like Dn, An, (An), (An) +, -(An), (d16,An), (d8,An,Xn), (bd,An,Xn), ([bd,An,Xn],od), and ([bd,An],Xn,od). Right table includes modes like (xxx).W, (xxx).L, # < data >, (d16,PC), (d8,PC,Xn), (bd,PC,Xn), ([bd,PC,Xn],od), and ([bd,PC],Xn,od).

PMOVE

Move to/from MMU Registers
(MC68030 only)

PMOVE

P-Register field—Specifies the memory management unit register.
 000—Translation Control Register
 010—Supervisor Root Pointer
 011—CPU Root Pointer

R/W field—Specifies the direction of transfer.
 0—Memory to memory management unit register.
 1—Memory management unit register to memory.

FD field—Disables flushing of the address translation cache on writes to memory management unit registers.
 0—Address translation cache is flushed.
 1—Address translation cache is not flushed.

Instruction Format:

MMU Status Register															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		
0	1	1	0	0	0	R/W	0	0	0	0	0	0	0	0	0

Instruction Fields:

Effective Address field—Specifies the memory location for the transfer. Control alterable addressing modes shown for supervisor root pointer register apply.

R/W field—Specifies the direction of transfer.
 0—Memory to MMU status register.
 1—MMU status register to memory.

NOTE

The syntax of assemblers for the MC68851 use the symbol PMMU status register for the MMU status register.



PMOVE

Move to/from MMU Registers
(MC68030 only)

PMOVE

Instruction Format:

TT Registers															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
											MODE		REGISTER		
0	0	0	P-REGISTER			R/ W	FD	0	0	0	0	0	0	0	0

Instruction Fields:

- Effective Address field—Specifies the memory location for the transfer. Control alterable addressing modes shown for supervisor root pointer register apply.
- P-Register field—Specifies the transparent translation register.
- 010—Transparent Translation Register 0
 - 011—Transparent Translation Register 1
- R/W field—Specifies the direction of transfer.
- 0—Memory to MMU status register.
 - 1—MMU status register to memory.
- FD field—Disables flushing of the address translation cache.
- 0—Address translation cache is flushed.
 - 1—Address translation cache does not flush.

PMOVE

Move to/from MMU Registers (MC68EC030)

PMOVE

Operation: If Supervisor State
Then (Source) → MRn or MRn → (Destination)

Assembler Syntax: PMOVE MRn, < ea >
PMOVE < ea > ,MRn

Attributes: Size = (Word, Long, Quad)

Description: Moves the contents of the source effective address to an access control register or moves the contents of an access control register to the destination effective address.

The instruction is a long-word operation for the access control registers (AC0 and AC1). It is a word operation for the access control unit status register (ACUSR).

Writing to the ACx registers enables or disables the access control register according to the E-bit written. If the E-bit = 1, the access control register is enabled. If the E-bit = 0, the register is disabled

Condition Codes:
Not affected.

ACUSR:
Not affected unless the ACUSR is specified as the destination operand.

Instruction Format:

ACUSR															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		
0	1	1	0	0	0	R/W	0	0	0	0	0	0	0	0	0

Instruction Fields:

Effective Address field—Specifies the memory location for the transfer.

R/W field—Specifies the direction of transfer.

0—Memory to ACUSR

1—ACUSR to memory



PMOVE

Move to/from MMU Registers
(MC68EC030)

PMOVE

NOTE

Assembler syntax for the MC68851 uses the symbol PMMU status register for the ACUSR; and for the MC68030, the symbols TT0 and TT1 for AC0 and AC1.

Instruction Format:

ACx Registers

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		
0	0	0	P-REGISTER			R/W	0	0	0	0	0	0	0	0	0

Instruction Fields:

Effective Address field—Specifies the memory location for the transfer.

P-Register field—Specifies the ACx register.

001—Access Control Register 0

011—Access Control Register 1

R/W field—Specifies the direction of transfer.

0—Memory to ACUSR

1—ACUSR to memory

PMOVE

Move PMMU Register (MC68851)

PMOVE

Operation: If Supervisor State
Then MC68851 Register → Destination
Or Source → MC68851 Register
Else TRAP

Assembler Syntax: PMOVE < PMMU Register > , < ea >
PMOVE < ea > , < PMMU Register >

Attributes: Size = (Byte, Word, Long, Double Long)

Description: The contents of the MC68851 register copies to the address specified by < ea > , or the data at < ea > copies into the MC68851 register.

The instruction is a quad-word operation for CPU root pointer, supervisor root pointer, and DMA root pointer registers. It is a long-word operation for the translation control register and a word operation for the breakpoint acknowledge control, breakpoint acknowledge data, access control, PMMU status, and PMMU cache status registers. PMOVE is a byte operation for the current access level, valid access level, and stack change control registers.

The following side effects occur when data is read into certain registers:

CPU Root Pointer—Causes the internal root pointer table to be searched for the new value. If there is no matching value, an entry in the root pointer table is selected for replacement, and all address translation cache entries associated with the replaced entry are invalidated.

Supervisor Root Pointer—Causes all entries in the address translation cache that were formed with the supervisor root pointer (even globally shared entries) to be invalidated.

DMA Root Pointer—Causes all entries in the address translation cache that were formed with the DMA root pointer (even globally shared entries) to be invalidated.

Translation Control Register—If data written to the translation control register attempts to set the E-bit and the E-bit is currently clear, a consistency check is performed on the IS, TIA, TIB, TIC, TID, and PS fields.



PMOVE Move PMMU Register (MC68851) PMOVE

PMMU Status Register: Not affected unless the PMMU status register is written to by the instruction.

Instruction Format 1:

PMOVE to/from TC, CRP, DRP, SRP, CAL, VAL, SCC, AC

Table with 16 columns (15-0) and 3 rows showing bit fields for PMOVE instruction format 1, including P-REGISTER, R/W, and EFFECTIVE ADDRESS MODE/REGISTER.

Instruction Fields:

Effective Address field—for memory-to-register transfers, any addressing mode is allowed as listed in the following table:

Two tables side-by-side showing Addressing Mode, Mode, and Register for memory-to-register transfers. The left table lists modes like Dn*, An*, (An), etc. The right table lists modes like (xxx).W, (xxx).L, # < data >, etc.

*PMOVE to CRP, SRP, and DMA root pointer not allowed with these modes

PMOVE

Move PMMU Register (MC68851)

PMOVE

For register-to-memory transfers, only alterable addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An*	001	reg. number:An	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	011	reg. number:An			
—(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	—	—
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	—	—

*PMOVE to CRP, SRP, and DMA root pointer not allowed with these modes

Register field—Specifies the MC68851 register.

- 000—Translation Control Register
- 001—DMA Root Pointer
- 010—Supervisor Root Pointer
- 011—CPU Root Pointer
- 100—Current Access Level
- 101—Valid Access Level
- 110—Stack Change Control Register
- 111—Access Control Register

R/W field—Specifies the direction of transfer.

- 0—Transfer < ea > to MC68851 register.
- 1—Transfer MC68851 register to < ea > .

Instruction Format 2:

PMOVE to/from BADx, BACx

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		
0	1	1	P-REGISTER			R/ W	0	0	0	0	NUM		0	0	



PMOVE

Move PMMU Register (MC68851)

PMOVE

Instruction Fields:

- Effective Address field—Same as format 1.
- P-Register field—Specifies the type of MC68851 register.
 - 100—Breakpoint Acknowledge Data
 - 101—Breakpoint Acknowledge Control
- R/W field—Specifies the direction of transfer.
 - 0—Transfer < ea > to MC68851 register
 - 1—Transfer MC68851 register to < ea >
- Num field—Specifies the number of the BACx or BADx register to be used.

Instruction Format 3:

PMOVE to/from PSR, from PCSR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		
0	1	1	P-REGISTER			R/ W	0	0	0	0	0	0	0	0	0

Instruction Fields:

- Effective Address field—Same as format 1.
- P Register field—Specifies the MC68851 register.
 - 000 — PMMU Status Register
 - 001 — PMMU Cache Status Register
- R/W field—Specifies direction of transfer.
 - 0—Transfer < ea > to MC68851 register.
 - 1—Transfer MC68851 register to < ea > (must be one to access PMMU cache status register using this format).

PRESTORE PMMU Restore Function (MC68851) PRESTORE

Operation: If Supervisor State
 Then MC68851 State Frame → Internal State, Programmer Registers
 Else TRAP

Assembler

Syntax: PRESTORE < ea >

Attributes: Unsized, Privileged

Description: The MC68851 aborts execution of any operation in progress. New programmer registers and internal states are loaded from the state frame located at the effective address. The first word at the specified address is the format word of the state frame, specifying the size of the frame and the revision number of the MC68851 that created it. The MC68020 writes the first word to the MC68851 restore coprocessor interface register, initiating the restore operation. Then it reads the response coprocessor interface register to verify that the MC68851 recognizes the format as valid. The format is invalid if the MC68851 does not recognize the frame size or the revision number does not match. If the format is invalid, the MC68020 takes a format exception, and the MC68851 returns to the idle state with its user visible registers unchanged. However, if the format is valid, then the appropriate state frame loads, starting at the specified location and proceeding up through the higher addresses.

The PRESTORE instruction restores the nonuser visible state of the MC68851 as well as the PMMU status register, CPU root pointer, supervisor root pointer, current access level, valid access level, and stack change control registers of the user programming model. In addition, if any breakpoints are enabled, all breakpoint acknowledge control and breakpoint acknowledge data registers are restored. This instruction is the inverse of the PSAVE instruction.

The current implementation of the MC68851 supports four state frame sizes:

- NULL:** This state frame is 4 bytes long, with a format word of \$0. A PRESTORE with this size state frame places the MC68851 in the idle state with no coprocessor or module operations in progress.
- IDLE:** This state frame is 36 (\$24) bytes long. A PRESTORE with this size state frame causes the MC68851 to place itself in an idle state with no coprocessor operations in progress and no breakpoints enabled. A module operation may or may not be in progress. This state frame restores the minimal set of MC68851 registers.



PRESTORE PMMU Restore Function (MC68851) PRESTORE

MID-COPROCESSOR: This state frame is 44 (\$2C) bytes long. A PRESTORE with this size frame restores the MC68851 to a state with a coprocessor operation in progress and no breakpoints enabled.

BREAKPOINTS ENABLED: This state frame is 76 (\$4C) bytes long. A PRESTORE with this size state frame restores all breakpoint registers, along with other states. A coprocessor operation may or may not be in progress.

PMMU Status Register: Set according to restored data.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	1	0	1	EFFECTIVE ADDRESS					
										MODE		REGISTER			

Instruction Fields:

Effective Address field—Specifies the source location. Only control or post-increment addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	011	reg. number:An			
—(An)	—	—			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	111	010
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	111	011
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	111	011

PSAVE

PMMU Save Function (MC68851)

PSAVE

Operation: If Supervisor State
Then MC68851 Internal State, Programmer
Registers → State Frame
Else TRAP

Assembler

Syntax: PSAVE < ea >

Attributes: Unsized, Privileged

Description: The MC68851 suspends execution of any operation that it is performing and saves its internal state and some programmer registers in a state frame located at the effective address. The following registers are copied: PMMU status, control root pointer, supervisor root pointer, current access level, valid access level, and stack change control. If any breakpoint is enabled, all breakpoint acknowledge control and breakpoint acknowledge data registers are copied. After the save operation, the MC68851 is in an idle state waiting for another operation to be requested. Programmer registers are not changed.

The state frame format saved by the MC68851 depends on its state at the time of the PSAVE operation. In the current implementation, three state frames are possible:

IDLE: This state frame is 36 (\$24) bytes long. A PSAVE of this size state frame indicates that the MC68851 was in an idle state with no coprocessor operations in progress and no breakpoints enabled. A module call operation may or may not have been in progress when this state frame was saved.

MID-COPROCESSOR: This state frame is 44 (\$2C) bytes long. A PSAVE of this size frame indicates that the MC68851 was in a state with a coprocessor or module call operation in progress and no breakpoints enabled.

BREAKPOINTS ENABLED: This state frame is 76 (\$4C) bytes long. A PSAVE of this size state frame indicates that one or more breakpoints were enabled. A coprocessor or module call operation may or may not have been in progress.

PMMU Status Register: Not affected



PSAVE

PMMU Save Function
(MC68851)

PSAVE

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	1	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		

Instruction Fields:

Effective Address field—Specifies the destination location. Only control or predecrement addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	—	—			
—(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	—	—
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	—	—

PScc

Set on PMMU unit Condition
(MC68851)

PScc

Operation: If Supervisor State
 Then If cc True
 Then 1s → Destination
 Else 0s → Destination
 Else TRAP

Assembler Syntax: PScc < ea >

Attributes: Size = (Byte)

Description: The specified MC68851 condition code is tested. If the condition is true, the byte specified by the effective address is set to TRUE (all ones); otherwise, that byte is set to FALSE (all zeros).

The condition code specifier cc may specify the following conditions:

Specifier	Description	Condition Field	Specifier	Description	Condition Field
BS	B set	000000	BC	B clear	000001
LS	L set	000010	LC	L clear	000011
SS	S set	000100	SC	S clear	000101
AS	A set	000110	AC	A clear	000111
WS	W set	001000	WC	W clear	001001
IS	I set	001010	IC	I clear	001011
GS	G set	001100	GC	G clear	001101
CS	C set	001110	CC	C clear	001111

PMMU Status Register: Not affected

PScc

Set on PMMU Condition
(MC68851)

PScc

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	1	EFFECTIVE ADDRESS					
										MODE		REGISTER			
0	0	0	0	0	0	0	0	0	0	MC68851 CONDITION					

Instruction Fields:

Effective Address field—Specifies the destination location. Only data alterable addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	011	reg. number:An			
—(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	—	—
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	—	—

MC68851 Condition field—Specifies the coprocessor condition to be tested. This field is passed to the MC68851, which provides directives to the main processor for processing this instruction.

PTEST

Test a Logical Address (MC68030 only)

PTEST

Operation: If Supervisor State
Then Logical Address Status → MMUSR
Else TRAP

Assembler Syntax: PTESTR FC, < ea > ,# < level >
PTESTR FC, < ea > ,# < level > ,An
PTESTW FC, < ea > ,# < level >
PTESTW FC, < ea > ,# < level > ,An

Attributes: Unsized

Description: This instruction searches the address translation cache or the translation tables to a specified level. Searching for the translation descriptor corresponding to the < ea > field, it sets the bits of the MMU status register according to the status of the descriptor. Optionally, PTEST stores the physical address of the last table entry accessed during the search in the specified address register. The PTEST instruction searches the address translation cache or the translation tables to obtain status information, but alters neither the used or modified bits of the translation tables nor the address translation cache. When the level operand is zero, only the transparent translation of either read or write accesses causes the operations of the PTESTR and PTESTW to return different results.

The < function code > operand is specified as one of the following:

1. Immediate—Three bits in the command word.
2. Data Register—The three least significant bits of the data register specified in the instruction.
3. Source Function Code (SFC) Register
4. Destination Function Code (DFC) Register

The effective address is the address to test. The < level > operand specifies the level of the search. Level 0 specifies searching the address translation cache only. Levels 1–7 specify searching the translation tables only. The search ends at the specified level. A level 0 test does not return the same MMU status register values as a test at a nonzero level number.

Execution of the instruction continues to the requested level or until detecting one of the following conditions:

- Invalid Descriptor
- Limit Violation
- Bus Error Assertion (Physical Bus Error)



PTEST

Test a Logical Address
(MC68030 only)

PTEST

The instruction accumulates status as it accesses successive table entries. When the instruction specifies an address translation cache search with an address register operand, the MC68030 takes an F-line unimplemented instruction exception.

If there is a parameter specification for a translation table search, the physical address of the last descriptor successfully fetched loads into the address register. A successfully fetched descriptor occurs only if all portions of the descriptor can be read by the MC68030 without abnormal termination of the bus cycle. If the root pointer's DT field indicates page descriptor, the returned address is \$0. For a long descriptor, the address of the first long word is returned. The size of the descriptor (short or long) is not returned and must be determined from a knowledge of the translation table.

Condition Codes:

Not affected.

MMUSR:

B	L	S		W	I	M			T				N
*	*	*	0	*	*	*	0	0	0	0	0	0	*

PTEST

Test a Logical Address (MC68030 only)

PTEST

The MMU status register contains the results of the search. The values in the fields of the MMU status register for an address translation cache search are given in the following table:

MMUSR Bit	PTEST, Level 0	PTEST, Levels 1–7
Bus Error (B)	This bit is set if the bus error bit is set in the ATC entry for the specified logical address.	This bit is set if a bus error is encountered during the table search for the PTEST instruction.
Limit (L)	This bit is cleared.	This bit is set if an index exceeds a limit during the table search.
Supervis or Violatio n (S)	This bit is cleared.	This bit is set if the S-bit of a long (S) format table descriptor or long format page descriptor encountered during the search is set and if the FC2-bit of the function code specified by the PTEST instruction is not equal to one. The S-bit is undefined if the I-bit is set.
Write Protecte d (W)	The bit is set if the WP-bit of the ATC entry is set. It is undefined if the I-bit is set.	This bit is set if a descriptor or page descriptor is encountered with the WP-bit set during the table search. The W-bit is undefined if the I-bit is set.
Invalid (I)	This bit indicates an invalid translation. The I-bit is set if the translation for the specified logical address is not resident in the ATC or if the B-bit of the corresponding ATC entry is set.	This bit indicates an invalid translation. The I-bit is set if the DT field of a table or a page descriptor encountered during the search is set to invalid or if either the B or L bits of the MMUSR are set during the table search.
Modified (M)	This bit is set if the ATC entry corresponding to the specified address has the modified bit set. It is undefined if the I-bit is set.	This bit is set if the page descriptor for the specified address has the modified bit set. It is undefined if I-bit is set.
Transparent (T)	This bit is set if a match occurred in either (or both) of the transparent translation registers (TT0 or TT1).	This bit is set to zero.
Number of Levels (N)	This 3-bit field is set to zero.	This 3-bit field contains the actual number of tables accessed during the search.

PTEST

Test a Logical Address (MC68030 only)

PTEST

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		
1	0	0	LEVEL			R/ W	A	REGISTER				FC			

Instruction Fields:

Effective Address field—Specifies the logical address to be tested. Only control alterable addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	—	—			
—(An)	—	—			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	—	—
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	—	—

Level field—Specifies the highest numbered level to be searched in the table. When this field contains 0, the A field and the register field must also be 0. The instruction takes an F-line exception when the level field is 0 and the A field is not 0.

R/W field—Specifies simulating a read or write bus cycle (no difference for MC68030 MMU).
0—Write
1—Read

A field—Specifies the address register option.
0—No address register.
1—Return the address of the last descriptor searched in the address register specified in the register field.

PTEST

Test a Logical Address (MC68030 only)

PTEST

Register field—Specifies an address register for the instruction. When the A field contains 0, this field must contain 0.

FC field—Function code of address to be tested.

10XXX — Function code is specified as bits XXX.

01DDD — Function code is specified as bits 2–0 of data register DDD.

00000 — Function code is specified as source function code register.

00001 — Function code is specified as destination function code register.



PTEST

Test a Logical Address
(MC68EC030)

PTEST

Operation: If Supervisor State
Then Logical Address Status → ACUSR
Else TRAP

Assembler Syntax: PTESTR FC, < ea >
PTESTW FC, < ea >

Attributes: Unsized

Description: This instruction searches the access control registers for the address descriptor corresponding to the < ea > field and sets the bit of the access control unit status register (ACUSR) according to the status of the descriptor.

The < function code > operand is specified in one of the following ways:

- 1. Immediate—Three bits in the command word.
- 2. Data Register—The three least significant bits of the data register specified in the instruction.
- 3. Source Function Code (SFC) Register
- 4. Destination Function Code (DFC) Register

The effective address is the address to test.

Condition Codes:

Not affected.

ACUSR:

x	x	x	0	x	x	x	0	0	AC	0	0	0	x	x	x
---	---	---	---	---	---	---	---	---	----	---	---	---	---	---	---

x = May be 0 or 1.

The AC-bit is set if a match occurs in either (or both) of the access control registers.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
											MODE		REGISTER		
1	0	0	0	0	0	R/ W	0	REGISTER			FC				

PTEST

Test a Logical Address (MC68EC030)

PTEST

Instruction Fields:

Effective Address field—Specifies the logical address to be tested. Only control alterable addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	—	—			
—(An)	—	—			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	—	—
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	—	—

R/W field—Specifies simulating a read or write bus cycle.

0—Write

1—Read

Register field—Specifies an address register for the instruction. When the A field contains 0, this field must contain 0.

FC field—Function code of address to be tested.

10XXX — Function code is specified as bits XXX.

01DDD — Function code is specified as bits 2–0 of data register DDD.

00000 — Function code is specified as source function code register.

00001 — Function code is specified as destination function code register.

NOTE

Assembler syntax for the MC68030 is PTESTR FC, < ea > ,#0
and PTESTW FC, < ea > ,#0.



PTEST

Test a Logical Address
(MC68040, MC68LC040)

PTEST

Operation: If Supervisor State
Then Logical Address Status → MMUSR; Entry → ATC
Else TRAP

Assembler Syntax: PTESTR (An)
PTESTW (An)

Attributes: Unsized

Description: This instruction searches the translation tables for the page descriptor corresponding to the test address in An and sets the bits of the MMU status register according to the status of the descriptors. The upper address bits of the translated physical address are also stored in the MMU status register. The PTESTR instruction simulates a read access and sets the U-bit in each descriptor during table searches; PTESTW simulates a write access and also sets the M-bit in the descriptors, the address translation cache entry, and the MMU status register.

A matching entry in the address translation cache (data or instruction) specified by the function code will be flushed by PTEST. Completion of PTEST results in the creation of a new address translation cache entry. The specification of the function code for the test address is in the destination function code (DFC) register. A PTEST instruction with a DFC value of 0, 3, 4, or 7 is undefined and will return an unknown value in the MMUSR.

- Execution of the instruction continues until one of the following conditions occurs:
- Match with one of the two transparent translation registers.
 - Transfer Error Assertion (physical transfer error)
 - Invalid Descriptor
 - Valid Page Descriptor

Condition Codes:
Not affected.

MMU Status Register:

PHYSICAL ADDRESS	B	G	U1	U0	S	CM	M		W	T	R
*	*	*	*	*	*	*	*	0	*	*	*

PTEST

Test a Logical Address (MC68040, MC68LC040)

PTEST

The MMUSR contains the results of the search. The values in the fields of the MMUSR for a search are:

Physical Address—This 20-bit field contains the upper bits of the translated physical address. Merging these bits with the lower bits of the logical address forms the actual physical address.

Bus Error (B)—Set if a transfer error is encountered during the table search for the PTEST instruction. If this bit is set, all other bits are zero.

Globally Shared (G)—Set if the G-bit is set in the page descriptor.

User Page Attributes (U1, U0)—Set if corresponding bits in the page descriptor are set.

Supervisor Protection (S)—Set if the S-bit in the page descriptor is set. This bit does not indicate that a violation has occurred.

Cache Mode (CM)—This 2-bit field is copied from the CM-bit in the page descriptor.

Modified (M)—Set if the M-bit is set in the page descriptor associated with the address.

Write Protect (W)—Set if the W-bit is set in any of the descriptors encountered during the table search. Setting of this bit does not indicate that a violation occurred.

Transparent Translation Register Hit (T)—Set if the PTEST address matches an instruction or data transparent translation register and the R-bit is set; all other bits are zero.

Resident (R)—Set if the PTEST address matches a transparent translation register or if the table search completes by obtaining a valid page descriptor.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	1	0	1	0	1	R/ W	0	1	REGISTER		

Instruction Fields:

R/W field—Specifies simulating a read or write bus transfer.

- 0—Write
- 1—Read

Register field—Specifies the address register containing the effective address for the instruction.



PTEST

Test a Logical Address
(MC68EC040)

PTEST

Operation: If Supervisor State
Then No Operation, Possibly Run Extraneous Bus Cycles
Else TRAP

Assembler Syntax: PTESTR (An)
PTESTW (An)

Attributes: Unsized

Description: This instruction must not be executed on an MC68EC040. This instruction may cause extraneous bus cycles to occur and may result in unexpected exception types.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	1	0	1	0	1	R/ W	0	1	REGISTER		

Instruction Fields:

- R/W field—Specifies simulating a read or write bus transfer.
- 0—Write
 - 1—Read
- Register field—Specifies the address register containing the effective address for the instruction.

PTEST

Get Information About Logical Address (MC68851)

PTEST

Operation: If Supervisor State
Then Information About Logical Address → PSTATUS
Else TRAP

Assembler PTESTR FC, < ea > ,# < level > ,(An)

Syntax: PTESTW FC, < ea > ,# < level > ,(An)

Attributes: Unsized

Description: If the E-bit of the translation control register is set, information about the logical address specified by FC and < ea > is placed in the PMMU status register. If the E-bit of the translation control register is clear, this instruction will cause a paged memory management unit illegal operation exception (vector \$39).

The function code for this operation may be specified as follows:

1. Immediate—The function code is four bits in the command word.
2. Data Register—The function code is in the lower four bits in the MC68020 data register specified in the instruction.
3. Source Function Code (SFC) Register—The function code is in the SFC register in the CPU. Since the SFC of the MC68020 has only three implemented bits, only function codes \$0D\$7 can be specified in this manner.
4. Destination Function Code (DFC) Register—The function code is in the DFC register in the CPU. Since the DFC of the MC68020 has only three implemented bits, only function codes \$0D\$7 can be specified in this manner.

The effective address field specifies the logical address to be tested.

The # < level > parameter specifies the depth to which the translation table is to be searched. A value of zero specifies a search of the address translation cache only. Values 1–7 cause the address translation cache to be ignored and specify the maximum number of descriptors to fetch.

NOTE

Finding an address translation cache entry with < level > set to zero may result in a different value in the PMMU status register than forcing a table search. Only the I, W, G, M, and C bits of the PMMU status register are always the same in both cases.

PTEST

Get Information About Logical Address (MC68851)

PTEST

Either PTESTR or PTESTW must be specified. These two instructions differ in the setting of the A-bit of the PMMU status register. For systems where access levels are not in use, either PTESTR or PTESTW may be used. U and M bits in the translation table are not modified by this instruction.

If there is a specified address register parameter, the physical address of the last successfully fetched descriptor is loaded into the address register. A descriptor is successfully fetched if all portions of the descriptor can be read by the MC68851 without abnormal termination of the bus cycle. If the DT field of the root pointer used indicates page descriptor, the returned address is \$0.

The PTEST instruction continues searching the translation tables until reaching the requested level or until a condition occurs that makes further searching impossible (i.e., a DT field set to invalid, a limit violation, or a bus error from memory). The information in the PMMU status register reflects the accumulated values.

PMMU Status Register:

Bus Error (B)—Set if a bus error was received during a descriptor fetch, or if $\langle \text{level} \rangle = 0$ and an entry was found in the address translation cache with its BERR bit set; cleared otherwise.

Limit (L)—Set if the limit field of a long descriptor was exceeded; cleared otherwise.

Supervisor Violation (S)—Set if a long descriptor indicated supervisor-only access and the $\langle \text{fc} \rangle$ parameter did not have bit 2 set; cleared otherwise.

Access Level Violation (A)—If PTESTR was specified, set if the RAL field of a long descriptor would deny access. If PTESTW was specified, set if a WAL or RAL field of a long descriptor would deny access; cleared otherwise.

Write Protection (W)—Set if the WP-bit of a descriptor was set or if a WAL field of a long descriptor would deny access; cleared otherwise.

Invalid (I)—Set if a valid translation was not available; cleared otherwise.

Modified (M)—If the tested address is found in the address translation cache, set to the value of the M-bit in the address translation cache. If the tested address is found in the translation table, set if the M-bit of the page descriptor is set; cleared otherwise.

PTEST

Get Information About Logical Address
(MC68851)

PTEST

- Gate (G)—If the tested address is found in the address translation cache, set to the value of the G-bit in the address translation cache. If the tested address is found in the translation table, set if the G-bit of the page descriptor is set; cleared otherwise.
- Globally Shared (C)—Set if the address is globally shared; cleared otherwise.
- Level Number (N)—Set to the number of levels searched. A value of zero indicates an early termination of the table search in the root pointer (DT = page descriptor) if the level specification was not zero. If the level specification was zero, N is always set to zero.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		
1	0	0	LEVEL			R/ W	A-REGISTER			FC					

PTEST

Get Information About Logical Address (MC68851)

PTEST

Instruction Fields:

Effective Address field—Specifies the logical address about which information is requested. Only control alterable addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	—	—			
—(An)	—	—			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	—	—
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	—	—

NOTE

The effective address field must provide the MC68851 with the effective address of the logical address to be tested, not the effective address describing where the PTEST operand is located. For example, to test a logical address that is temporarily stored on the system stack, the instruction PTEST [(SP)] must be used since PTEST (SP) would test the mapping of the system stack (i.e., the effective address passed to the MC68851 is the effective address of the system stack, not the effective address formed by the operand located on the top of the stack).

PTEST

Get Information About Logical Address (MC68851)

PTEST

Level field—Specifies the depth to which the translation table should be searched.

R/W field—Specifies whether the A-bit should be updated for a read or a write.

1—Read

0—Write

A-Register field—Specifies the address register in which to load the last descriptor address.

0xxx — Do not return the last descriptor address to an address register.

1RRR — Return the last descriptor address to address register RRR.

NOTE

When the PTEST instruction specifies a level of zero, the A-register field must be 0000. Otherwise, an F-line exception is generated.

FC field—Function code of address to test.

1DDDD — Function code is specified as four bits DDDD.

01RRR — Function code is contained in CPU data register RRR.

00000 — Function code is contained in CPU source function code register.

00001 — Function code is contained in CPU destination function code register.



PTRAPcc

TRAP on PMMU Condition
(M68851)

PTRAPcc

Operation: If Supervisor State
Then If cc True
Then TRAP
Else TRAP

Assembler PTRAPcc
Syntax: PTRAPcc.W # < data > PTRAPcc.L # < data >

Attributes: Unsized or Size = (Word, Long)

Description: If the selected MC68851 condition is true, the processor initiates exception processing. The vector number is generated referencing the cpTRAPcc exception vector; the stacked program counter is the address of the next instruction. If the selected condition is not true, no operation is performed, and execution continues with the next instruction. The immediate data operand is placed in the next word(s) following the MC68851 condition and is available for user definition to be used within the trap handler. Following the condition word, there may be a user-defined data operand, specified as immediate data, to be used by the trap handler.

The condition specifier cc may specify the following conditions:

Specifier	Description	Condition Field	Specifier	Description	Condition Field
BS	B set	000000	BC	B clear	000001
LS	L set	000010	LC	L clear	000011
SS	S set	000100	SC	S clear	000101
AS	A set	000110	AC	A clear	000111
WS	W set	001000	WC	W clear	001001
IS	I set	001010	IC	I clear	001011
GS	G set	001100	GC	G clear	001101
CS	C set	001110	CC	C clear	001111

PMMU Status Register: Not affected

PTRAPcc

TRAP on PMMU Condition
(M68851)

PTRAPcc

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	1	1	1	1	OPMODE		
0	0	0	0	0	0	0	0	0	0	MC68851 CONDITION					
16-BIT OPERAND OR MOST SIGNIFICANT WORD OF 32-BIT OPERAND (IF NEEDED)															
LEAST SIGNIFICANT WORD OF 32-BIT OPERAND (IF NEEDED)															

Instruction Fields:

- Opmode field—Selects the instruction form.

010 — Instruction is followed by one operand word.
011 — Instruction is followed by two operand words.
100 — Instruction has no following operand words.
- MC68851 Condition field—Specifies the coprocessor condition to be tested. This field is passed to the MC68851, which provides directives to the main processor for processing this instruction.



PVALID

Validate a Pointer
(MC68851)

PVALID

Operation: If (Source AL Bits) → (Destination AL Bits)
Then TRAP

Assembler Syntax: PVALID VAL, < ea >
PVALID An, < ea >

Attributes: Size = (Long)

Description: The upper bits of the source, VAL or An, compare with the upper bits of the destination, < ea > . The ALC field of the access control register defines the number of bits compared. If the upper bits of the source are numerically greater than (less privileged than) the destination, they cause a memory management access level exception. Otherwise, execution continues with the next instruction. If the MC field of the access control register = 0, then this instruction always causes a paged memory management unit access level exception.

PMMU Status Register: Not affected.

Instruction Format 1:

VAL Contains Access Level to Test Against

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE		REGISTER			
0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0

PVALID

Validate a Pointer (MC68851)

PVALID

Instruction Field:

Effective Address field—Specifies the logical address to be evaluated and compared against the valid access level register. Only control alterable addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register
Dn	—	—
An	—	—
(An)	010	reg. number:An
(An) +	—	—
—(An)	—	—
(d ₁₆ ,An)	101	reg. number:An
(d ₈ ,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
([bd,An,Xn] ,od)	110	reg. number:An
([bd,An],Xn ,od)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
# < data >	—	—
(d ₁₆ ,PC)	—	—
(d ₈ ,PC,Xn)	—	—
(bd,PC,Xn)	—	—
([bd,PC,Xn] ,od)	—	—
([bd,PC],Xn ,od)	—	—



PVALID

Validate a Pointer (MC68851)

PVALID

Instruction Format 2:

Main Processor Register Contains Access Level to Test Against

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		
0	0	1	0	1	0	0	0	0	0	0	0	0	REGISTER		

Instruction Fields:

Effective Address field—Specifies the logical address to be evaluated and compared against specified main processor address register. Only control alterable addressing modes can be used as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	# < data >	—	—
(An) +	—	—			
—(An)	—	—			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	—	—
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn] ,od)	110	reg. number:An	([bd,PC,Xn] ,od)	—	—
([bd,An],Xn ,od)	110	reg. number:An	([bd,PC],Xn ,od)	—	—

NOTE

The effective address field must provide the MC68851 with the effective address of the logical address to be validated, not the effective address describing where the PVALID operand is located. For example, to validate a logical address that is temporarily stored on the system stack, the instruction PVALID VAL,[(SP)] must be used since PVALID VAL,(SP) would validate the mapping on the system stack (i.e., the effective address passed to the MC68851 is the effective address of the system stack, not the effective address formed by the operand located on the top of the stack).

Register field—Specifies the main processor address register to be used in the compare.

RESET

Reset External Devices
(M68000 Family)

RESET

Operation:

If Supervisor State
Then Assert RESET ($\overline{RSTO}$, MC68040 Only) Line
Else TRAP

Assembler
Syntax:

RESET

Attributes:

Unsize

Description:

Asserts the $\overline{RSTO}$ signal for 512 (124 for MC68000, MC68EC000, MC68HC000, MC68HC001, MC68008, MC68010, and MC68302) clock periods, resetting all external devices. The processor state, other than the program counter, is unaffected, and execution continues with the next instruction.

Condition Codes:

Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	1	1	1	0	0	1	1	1	0	0	0	0



RTE

Return from Exception
(M68000 Family)

RTE

Operation: If Supervisor State
Then (SP) → SR; SP + 2 → SP; (SP) → PC; SP + 4 → SP; Restore State and Deallocate Stack According to (SP)
Else TRAP

Assembler Syntax: RTE

Attributes: Unsized

Description: Loads the processor state information stored in the exception stack frame located at the top of the stack into the processor. The instruction examines the stack format field in the format/offset word to determine how much information must be restored.

Condition Codes:
Set according to the condition code bits in the status register value restored from the stack.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	1	1	1	0	0	1	1	1	0	0	1	1

Format/Offset Word (in Stack Frame):

MC68010, MC68020, MC68030, MC68040, CPU32

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FORMAT				0	0	VECTOR OFFSET									

Format Field of Format/Offset Word:

Contains the format code, which implies the stack frame size (including the format/offset word). For further information, refer to **Appendix B Exception Processing Reference**.

STOP

Load Status Register and Stop
(M68000 Family)

STOP

Operation: If Supervisor State
Then Immediate Data → SR; STOP
Else TRAP

Assembler Syntax: STOP # < data >

Attributes: Unsized

Description: Moves the immediate operand into the status register (both user and supervisor portions), advances the program counter to point to the next instruction, and stops the fetching and executing of instructions. A trace, interrupt, or reset exception causes the processor to resume instruction execution. A trace exception occurs if instruction tracing is enabled (T0 = 1, T1 = 0) when the STOP instruction begins execution. If an interrupt request is asserted with a priority higher than the priority level set by the new status register value, an interrupt exception occurs; otherwise, the interrupt request is ignored. External reset always initiates reset exception processing.

Condition Codes:
Set according to the immediate operand.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	1	1	1	0	0	1	1	1	0	0	1	0
IMMEDIATE DATA															

Instruction Fields:
Immediate field—Specifies the data to be loaded into the status register.

